

Normativa del Proyecto

Autor: Equipo SomnGuard

Fecha de creación: 2026-04-12

Estado: Aprobado

1. Objetivo

Este documento establece las normas, políticas y estándares que regirán la elaboración, revisión, aprobación y publicación de la documentación, código y artefactos del proyecto SomnGuard.

2. Alcance

Aplica a todo el equipo del proyecto:

- Documentación técnica y funcional
- Código fuente y configuraciones
- Ramas de control de versiones (Git)
- Entregables y artefactos de despliegue

3. Convenciones generales

Lenguaje y formato

- **Documentación:** Español neutro. El contenido documental se redacta en español, evitando jerga local y explicando tecnicismos cuando sea necesario.
- **Código y commits:** Inglés. Los nombres de ramas, mensajes de commit, identificadores técnicos y comentarios de código se escriben en inglés.
- **Documentación:** Preferentemente en Markdown (`.md`). Cuando se necesite una versión de consulta sin vista previa, también puede exportarse a PDF; si se requiere un formato editable adicional, se admiten archivos tipo documento como `.docx`.
- **Diagramas:** Solo en Mermaid (`.mmd`) por ahora, con exportación a `.png` para visualización o entrega.
- **Codificación:** UTF-8 sin BOM, salvo que una herramienta o entorno específico indique otra necesidad.
- **Fechas:** Formato `YYYY-MM-DD`

4. Nomenclatura y estilos

Archivos y carpetas

- **Nombres de archivos:** minúsculas, guiones para separar palabras
Ejemplo: `plan_pruebas.md`, `arquitectura_inicial.md`

Código

- **Lenguaje principal:** C#
- **C#:** `camelCase` para variables y métodos; `PascalCase` para clases, interfaces y tipos
- **Constantes:** `MAYÚSCULAS_CON_GUIONES`

Documentación

- **Títulos:** H1 para documentos principales, H2/H3 para secciones
- **Listas:** numeradas para procesos, viñetas para atributos
- **Énfasis: negrita** para términos clave, **código** para referencias técnicas

Diagramas

- Usar convenciones estándar (UML)

5. Estructura documental

La documentación se organiza en carpetas temáticas:

```
00_Gestion_Proyecto/      # Actas, cronogramas, normativas
01_Requisitos/            # SRS, casos de uso, requisitos funcionales
02_Arquitectura_y_Diseño/ # Arquitectura, diagramas, patrones
03_Datos/                 # Modelo de datos, diccionarios, esquemas
04_UX_UI/                 # Prototipos, wireframes, guías
05_Desarrollo/            # Código fuente, guías técnicas
06_Pruebas_y_Calidad/     # Planes de test, reportes
07_Despliegue_y_Operaciones/ # Scripts, configuración, operaciones
08_Anexos/                # Recursos adicionales
```

Encabezado mínimo en documentos

Todo documento debe incluir al inicio:

```
# [Título del documento]

**Autor:** [Nombre/Rol]
**Fecha de creación:** YYYY-MM-DD
**Última actualización:** YYYY-MM-DD

---
```

6. Control de versiones de documentos

Política de versionado

- **Sin versionado formal** mientras el proyecto está en fase inicial.
- Una vez estable, adoptar **vA.B** (ej: **v1.0**, **v1.1**).

Historial de cambios

Incluir tabla en cada documento importante:

Fecha	Estado	Autor	Cambios
2026-04-12	Aprobado	Cristian Javier Palma Sotto	Creación inicial

7. Control de código y ramas

Estructura de ramas

- **main** — rama de producción, siempre estable y desplegable
- **develop** — rama de integración, para cambios integrados
- **qa** — rama de QA/pruebas antes de pasar a main
- **HU-###-descripcion-(ambiente: dev, qa, main)** — desarrollo de historias de usuario
- **BUG-###-descripcion(ambiente: dev, qa, main)** — correcciones de bugs

Reglas de merge

- Todo cambio debe realizarse mediante **Pull Request (PR)**
- **Mínimo requisitos para merge:**
 - 1 aprobación de revisión técnica (Líder Técnico)
 - Pruebas locales exitosas
 - Sin conflictos
- **Merge a main** requiere aprobación explícita del Líder Técnico
- **Merge a develop** requiere aprobación de al menos 1 revisor

8. Mensajes de commit

Formato: Conventional Commits

```
<tipo>(<ámbito>): <descripción breve>

<cuerpo opcional>

<referencias opcionales>
```

Tipos de commit

- **feat** — nueva característica
- **fix** — corrección de bug
- **refactor** — refactorización de código
- **test** — adición/modificación de tests
- **docs** — cambios en documentación
- **chore** — tareas (dependencias, configuración)

Ejemplos

```
feat(auth): agregar login OAuth2
```

Implementa autenticación con OAuth2 para conductores.

Refs: #HU-001

`fix(sensor): corregir lectura de cámara`

Ajusta timeout de captura de imágenes en Raspberry Pi.

Refs: #BUG-042

9. Revisión y aprobación

Revisión de código

- **Revisor designado:** Líder Técnico (Cristian Javier Palma Sotto)
- **Criterios:** funcionalidad, calidad, seguridad, pruebas

Revisión de documentación

- **Documentos críticos** (Arquitectura, SRS): Líder Técnico + Product Owner (cuando aplique)
- **Documentación funcional:** Responsable de área + Líder Técnico

Aprobación final

- **Documentos finales:** Líder Técnico Cristian Javier Palma Sotto
- **Estado:** Marcar como **Aprobado** en el encabezado y registrar en acta

10. Seguridad y privacidad

Políticas de seguridad

- **No incluir en el repositorio:**
 - Credenciales (contraseñas, tokens, claves privadas)
 - Información sensible o confidencial
 - Datos personales reales
- **Alternativas:**
 - Usar **.env** local (no versionado)
 - Referenciar a bóveda de secretos
 - Usar datos ficticios en ejemplos

Revisión de dependencias

- Revisar dependencias críticas contra vulnerabilidades antes de integrar
- Mantener listos de dependencias actualizados
- Reportar CVE y planificar updates

11. Gestión de licencias

Cumplimiento de licencias

- Respetar licencias de software de terceros
- Validar compatibilidad con licencia del proyecto
- Evitar dependencias con licencias restrictivas (ej: GPL) sin evaluación

12. Entregables y artefactos

Tipos de entregables

Tipo	Descripción	Ubicación
Código fuente	Repositorio Git completo	/05_Desarrollo
Documentación técnica	Manuales, guías, API docs	/02_Arquitectura_y_Diseño, /05_Desarrollo
Documentación funcional	SRS, casos de uso	/01_Requisitos
Datos	Modelos, esquemas	/03_Datos
Reportes de pruebas	Test reports, coverage	/06_Pruebas_y_Calidad
Binarios/Releases	Versiones empaquetadas	/releases/v*.*.*.zip

Versionado de releases

Formato: `somnguard-v{MAJOR}.{MINOR}.{PATCH}`

Ejemplo: `somnguard-v1.0.0.zip`

13. Gestión de cambios

Para cambios menores

- Crear PR directamente
- Incluir descripción clara del cambio
- Solicitar revisión

Para cambios mayores o críticos

- Documentar propuesta de cambio (RFC - Request For Change)
- Incluir: objetivo, alcance, impacto, plan de pruebas
- Revisar y aprobar antes de implementar
- Registrar en acta si procede

14. Auditoría y trazabilidad

Mecanismos de registro

- **Actas de reuniones:** `/00_Gestion_Proyecto/03_actas/` — decisiones, acuerdos, compromisos
- **Commits:** Git log — cambios de código con referencias a tickets

- **Pull Requests:** GitHub/GitLab — discusiones técnicas, revisiones

Decisiones arquitectónicas

- Cuando se tome una decisión de arquitectura importante, documentar brevemente:
 - Opción elegida
 - Alternativas consideradas
 - Justificación
 - Fecha
 - Responsable

15. Contactos del proyecto

Rol	Nombre
Líder Técnico	Cristian Javier Palma Sotto
Desarrollador	Juan Carlos Jurado Castañeda
Desarrollador	Johan Steven Rodriguez Charry
Desarrollador	Brayan Alberto Perdomo

16. Anexos

Plantillas y referencias:

- Acta de reuniones: [/00_Gestion_Proyecto/03_actas/](#)
- SRS / Requisitos: [/01_Requisitos/](#)
- Template de PR: A definir en repositorio

17. Glosario

Término	Significado
PR	Pull Request — solicitud de cambio de código
RFC	Request For Change — solicitud de cambio mayor
HU	Historia de Usuario
BUG	Defecto/error reportado
SRS	Documento de Especificación de Requisitos
Git	Sistema de control de versiones distribuido

Última actualización: 2026-05-03